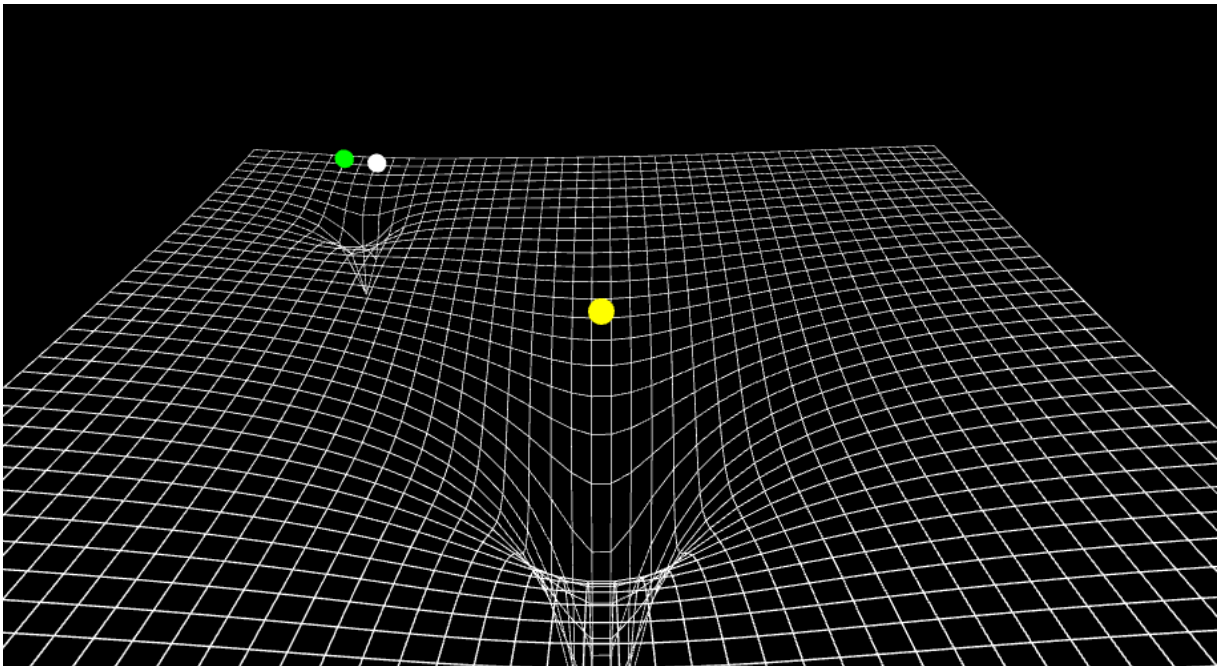


NEXT SUKKERTOPPEN GYMNASIUM
2.W

STUDIE VALGFAGS CASE - SVC
PROGRAMMERING B & FYSIK A

Simulering af gravitation
og visualisering af rumtid



Khizar Irshad Chaudhry

23. marts 2025

**RUMMET DIKTERER LEGEMERS BEVÆGELSER.
LEGEMER DIKTERER RUMMETS KRUMNINGER.**

RESUMÉ

Denne projekt omhandler simuleringen af gravitationelle interaktioner mellem legemer.

Programmet tillader brugeren til at tilføje legemer med selvvalgte positioner, starthastigheder og masser, og visualiserer deres bevægelser under vpårikning af gravitationen, samt dens påvirkning på rumtiden. Simuleringen opdaterer dynamisk legemernes position og hastighed baseret på Newtons gravitationslov. Projektet blev oprindeligt startet som et hobbyprojekt og udviklet til et studieprojekt i Programmering B (studie valgs projekt - SVC).

Indhold

Resumé	iii
Indhold	iv
Forord	v
Forord	v
1 Indledning	1
2 Kravspecifikation	2
3 Computational Thinking i Programmet	4
3.1 Pattern Recognition (PR)	4
3.2 Abstraction	5
3.3 Decomposition	6
3.4 Algorithm Design	7
4 Udviklingsprocessen	8
4.1 Intial commit & P5.js (WEBGL)	8
4.2 Tilføjet legeme klasse (OOP)	8
4.3 Tilføjet gitter og lavet liste til legemer	9
4.4 Tilføjet tyngdekraften mellem legemer & krumning af rumtid	9
4.5 Tilføjet et orbital i systemet	9
4.6 Tilføjet reset kamera knap	9
4.7 Tilføjet funktion til at tilføje legemer	10
4.8 Vektorpile til hastighed og kraft	10
5 Dokumentation af Programmet	11
5.1 Hierarki chart for programmet	11
5.2 Pseudokode	12
5.2.1 Pseudokode for Tilføjelse af Legeme	12
5.2.2 Pseudokode for opdatering af legemer	12
5.3 Kodestykker med indlejrede kommentarer	13
5.3.1 Koden for opdatering af legemerne	13
5.3.2 Visualisering af legemer og vektorpile	14
5.4 Dokumentation af egne funktioner/klasse	15
5.5 Selveste koden & programmet	16
5.6 Flowchart repræsentation af programmet	16
6 Test	19
7 Diskussion	20
8 Konklusion	21

Forord

Denne software var ikke det første, jeg begyndte på i forbindelse med SVC-opgaven. Jeg havde faktisk allerede lavet et program omkring lys og Snells lov ([som kan findes her](#)).

Men det var ikke tilstrækkeligt interessant, spændende eller udfordrende, hvad jeg havde lavet indtil videre. Derfor begyndte jeg på dette projekt, som oprindeligt var et hobbyprojekt, men som med tiden blev mit SVC-projekt. Jeg er klar over, at denne rapport er langt mere omfattende end opgavens oprindelige krav. Derfor har jeg ikke nogen forventninger om, at hele rapporten bliver læst igennem og vurderet. Dette projekt er en sammensmeltning af mit hobbyarbejde omkring astronomiske legemer, sorte huller og relativitet, samt min skoleopgave i Programmering B (Studievalgs Case - SVC).

Jeg ser frem til at arbejde videre på dette projekt og denne dokumentation*.

* Dokumentation er skrevet vha. L^AT_EX

Tyngdekraft. En kraft vi alle er bekendte omkring, og oplever hverdag¹. Ifølge Isaac Newtons (1642-1727) lov om universel gravitation er *tyngdekraft* en kraft, der mellem to legemer er propotionel med deres distance og hhv. masser². Dette stemmer dog ikke ens med Albert Einsteins (1879-1955) syn på tyngdekraft. Einsteins generelle relativitetsteori beskriver tyngdekraften som en krumning af rumtid³. Alle legemer følger rumtidens baner, samt krummer alle legemer også rumtiden. Legemere med størrer masse krummer rumtiden mere end legemer med mindre masse. Det er netop derfor, at legemer som planeter har eliptiske kredsløb omkring større legemer, der krummer rumtid (se Figur 1.1).

Ved hjælp af både Newtons love og Einsteins teorier, kan legemers bevægelser simuleres og visualiseres. Det er dog meget kompleks simulere Einsteins relavistiske tilgang⁴, og kræver avanceret viden indenfor fysik. Derfor kan en forsimpeling benytets for visualisering af rumtiden. Dette leder til problemstillingen:

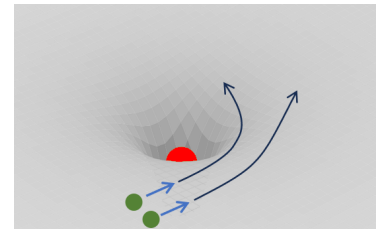
Hvordan kan legemeres hhv. tyngdekraft simuleres og visualiseres vha. programmering?

Til det kan Javascript og P5.js bruges. For en overskuelig implemantation af tyngdekraften er det klart bedre at benytte Newtonsk-tyngdekraft, fremfor relativistisk-tyngdekraft. Dog er det stadig en god idé at visualisere rumtiden og dens krumning. Yderligere kunne man evt. benytte object-oriented programmering, da programmet skal kunne simulere flere legemere. Disse tanker kan ses i det dokumenterede mindmap senere i Figur 3.3 i sektion 3.3, *Decomposition*.

1: Om det så er vores fødder fortsat sat til jords, eller jorden fortsat sat i kredsløb.

$$2: F = G \cdot \frac{m_1 \cdot m_2}{r^2}$$

3: Rumtid beskrives som den fjerde-dimensionelle kontinuum, der udgøres af rum (de tre dimensioner) og tid. Derfor må et legeme beskrives med en rumtids-koordinate (x, y, z, t) .



Figur 1.1: En visualisering af rumtidens krumning grundet store masser (røde), samt påvirkningen på andre legemer (grøn).*

4: Dette kræver at løse Einsteins *field equations*: $G_{\mu\nu} + \Lambda g_{\mu\nu} = kT_{\mu\nu}$

* Ukendt, https://javalab.org/en/general_relativity_en/ (Sidst besøgt: 22. marts 2025)

2 Kravspecifikation

Ud fra tankerne fra indledningen og det dannet mindmap (se Figur 3.3), kan nogle kravspecifikationer opstilles til programmet. Kravene i kravspecifikationen kan opdeles i *need-to-have* og *nice-to-have*¹. Her er programmets kravspecifikationer der er *need-to-have*, og er implementeret i den færdige udgave af programmet:

1: Altså hhv. krav der er nødvendige at have og krav der kan implementeres, hvis der er resourcer til det.

1. Simulering af gravitation

- ▶ Programmet skal simulere tyngdekraften mellem flere legemer
- ▶ Tyngdekraften skal beregnes vha. Newtons gravitationslov $F = G \cdot \frac{m_1 \cdot m_2}{r^2}$
- ▶ Programmet skal tage højde for legemernes position, masse og hastighed
- ▶ Tyngdekraften skal påvirke legemernes acceleration², som opdaterer de hhv. hastigheder og positioner.

2: Hvor accelerationen kan beregnes vha. kraftloven $F = ma$.

2. Interaktivitet

- ▶ Brugeren skal kunne tilføje legemer, givet en position, masse og starthastighed vha. inputfelter
- ▶ Skifte kamera-vinkel og positionen

3. Visualisering

- ▶ Programmet skal vise legemerene
- ▶ Programmet skal vise rumtids-gitteret, og krumningen i rumtids-gitteret.
- ▶ Krumningen i rumtids-gitteret skal være afhængig af legemeres masse, og distancen til legemerene.

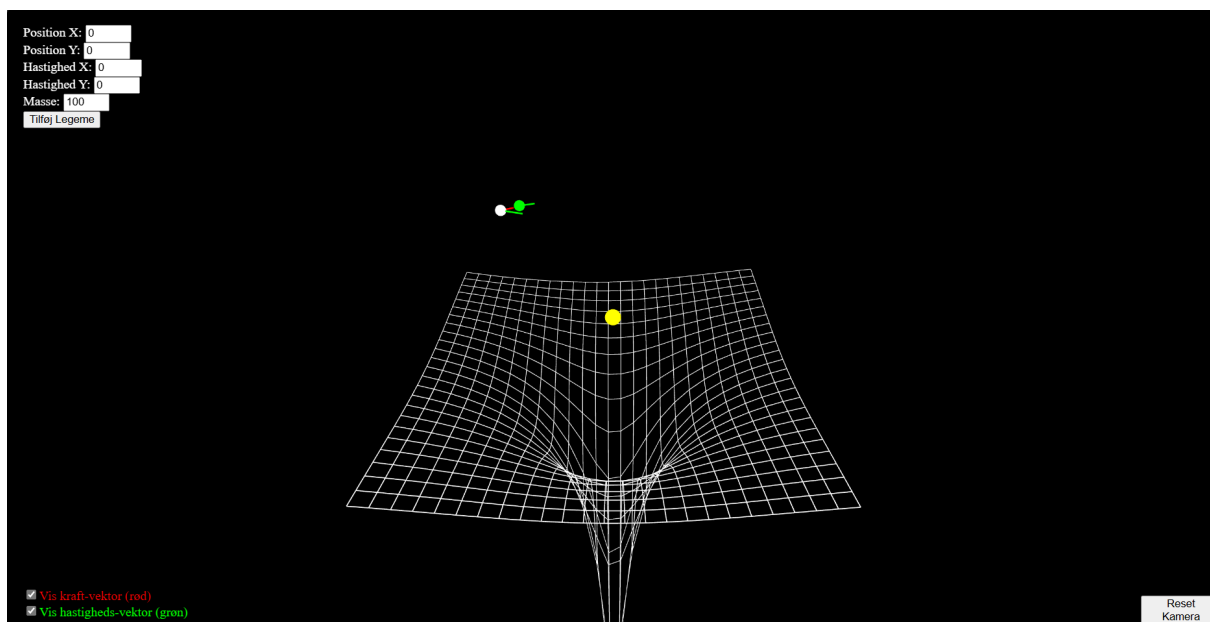
4. Teknologien

- ▶ JavaScript og p5.js benyttes til programmet
- ▶ Programmet skal kunne køre i moderne web browsere

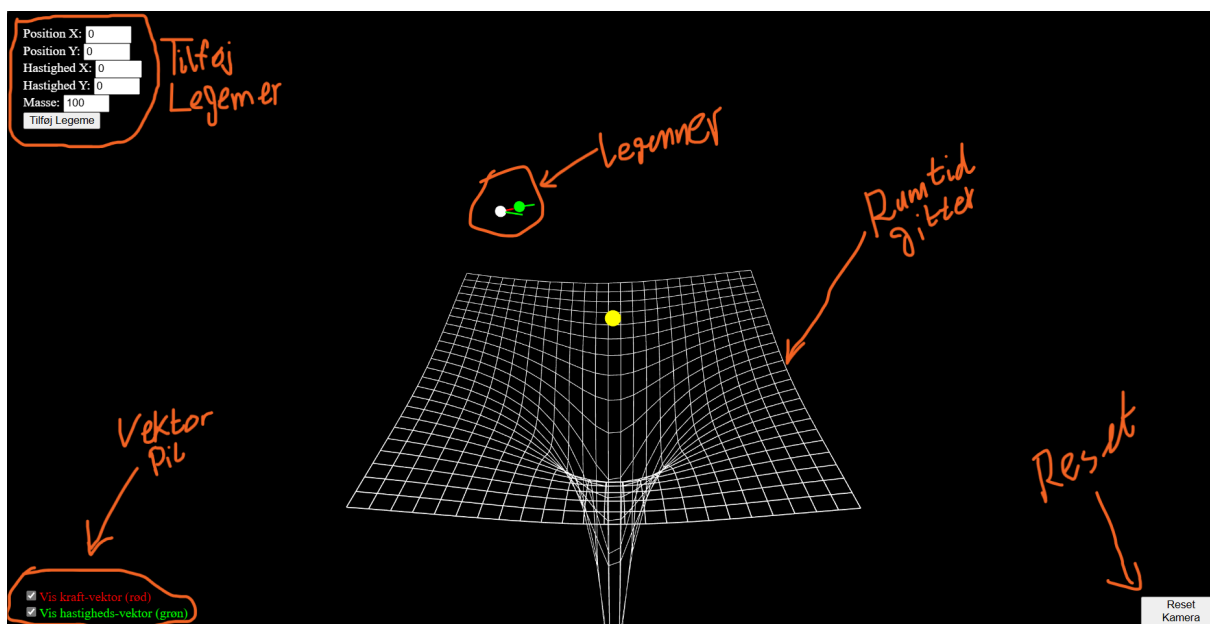
Programmets færdige udgave er yderligere forbedret ved at implementere nogle *nice-to-have* krav som:

1. Interaktivitet

- Funktioner, der giver brugeren mulighed for at vælge om hastigheds- og kraftvektorer skal vises på legemerne.
- En knap til at nulstille kamerapositionen

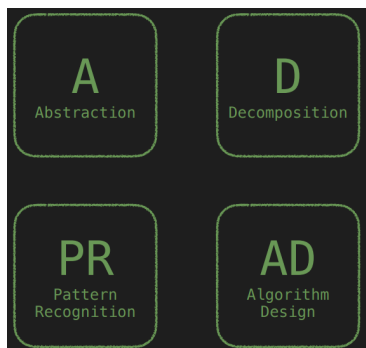


På figuren ovenfor ses et screenshot af programmet, hvor alle kravene fra kravspecifikationen inkluderet. Nogle af kravene er kan ses peget ud på billedet her:



3

Computational Thinking i Programmet



Figur 3.1: De fire karakteristiske aspekter, der definere computational thinking.*

Datamatikere har en helt særlig process og tankegang når de løser problemer. De omformulere problemerne således, at løsninger kan repræsenteres som *computational* trin. I andre ord benytter de sig af metodikken *computational thinking* (CT). CT består af at dele problemet i fire centrale aspekter: abstraction (A), decomposition (D), pattern recognition (PR) og algorithm design (AD) (se Figur. 3.1)

I det pågældende program, der simulerer gravitationen mellem legemer, anvendes de fire grene af computational thinking til at strukturere og udvikle programmet.

3.1 Pattern Recognition (PR)

Mønster genkendelse er essentiel for at udvikle et program. Det er igennem disse mønstre programmet kodes. I dette program er der gentagne og forudsigelige mønstre, der simuleres. Dette inkluderer:

► Gravitationsinteraktioner

Det genkendes at gravitationskraften mellem alle legemer følger Newtons lov om universel gravitation⁰:

$$F = G \cdot \frac{m_1 \cdot m_2}{r^2} \quad (3.1)$$

For et system med j legemer, skal kraften F_i på legemet i være afhængig af alle de andre legemer j og deres masse m_j samt afstanden r_{ij} mellem legeme i, j :

$$F_i = \sum_{j \neq i} G \cdot \frac{m_i m_j}{r_{ij}^2}$$

Dette gælder for alle legemer, og derfor kan betragtes som et gentagende mønster. I programmet kan denne mønster evt. implementeres som en *for-løkke* og de hhv. krafter kan beregnes.

0: Hvor:

- F er den udøvede kraft på legemet
- G er gravitationskonstanten,
- m_1, m_2 er legemers hhv. masser,
- r er afstanden mellem legemernes centrum.

* Petersen, Anders Roland. *Powerpoint: Software Development*

► Bevægelsesmøntre

Legemers bevægelser følger også et mønster, der kan generaliseres. Legemers hhv. acceleration under påvirkning af gravitationen kan bestemmes vha. kraftloven (Newtons 2. lov)⁰:

$$F = ma$$

$$\Updownarrow a = \frac{F}{m}$$

Givet accelerationen til en tid t kan legemernes hastighed til tiden t bestemmes vha. integration:

$$v(t) = \int a(t) dt + v_0$$

Dernæst kan positionen $p(t)$ også bestemmes ved at integrere hastigheden $v(t)$:

$$p(t) = \int v(t) dt + p_0$$

Dette gælder for alle legemer, og derfor er et implementerbar mønster, der kan opdatere legemeres positioner i programmet vha. accelerationen forudsat af tyngdekraften.

► Legemer

Alle legemer følger den samme fysiske mønster, da de alle er kugler, og har fysiske egenskaber som masse, hastighed, acceleration, position og farve¹.

0: Hvor:

- F er kraften fra Ligning (3.1),
- m er legemets masse,
- a er legemets acceleration.

1: se Figur 3.2 på næste side

3.2 Abstraction

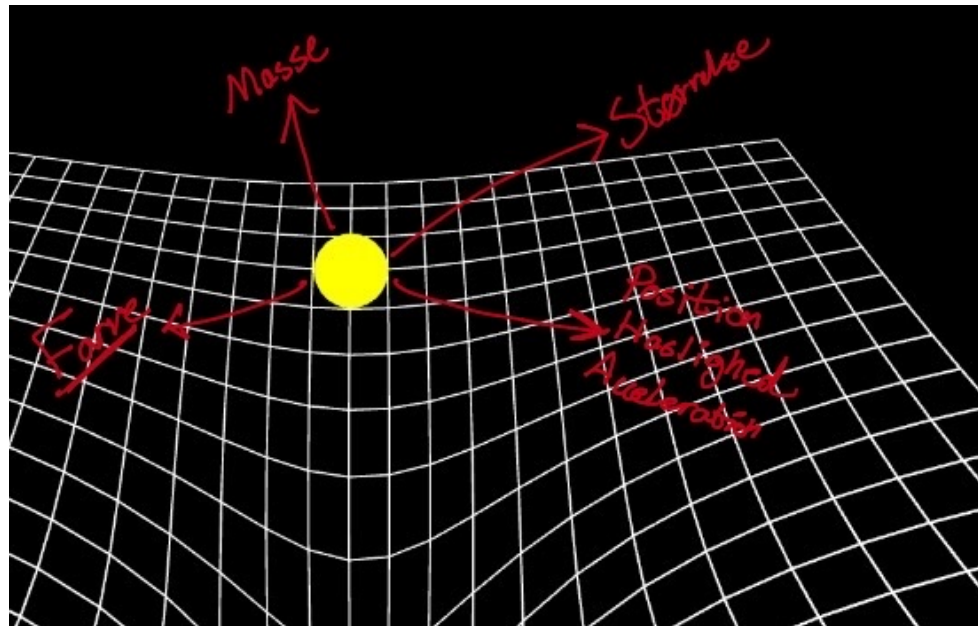
Abstraktion indebære at forenkle problemet, fjerne irrelevante detaljer og kun fokusere på de væsentlige aspekter. I dette program er abstraktion blevet anvendt til at forenkle tyngdekraften og rumtidens komplekse karakter, og kompaktisere dele af simuleringen.

I programmet anvendes en newtonsk tilgang for at beregne legemernes tyngdekraft, fremfor et relativistisk tilgang. Denne abstraktion var nødvendig, da det er unødvendig svært at implementere et relativistisk tyngdekraft². Yderligere abstrahere programmet den visuelle rumtidens krumning med $-m \cdot \frac{20}{d+10}$, fremfor en realistisk krumning. Disse abstraktioner forenkler programmet og inddrager urealistiske elemente. Dog hjælper det med at opnå programmets formål, om at simulere tyngdekraften og visualisere rumtidens

2: Hvilket kræver at løse Einsteins field equations.

krumning.

Programmets kode anvender object-oriented programming (OOP), som er med til abstrahere legemet som en *klasse*. Legemet i simuleringen er fysiske objekter (kugler), der har en position, hastighed, acceleration, masse og farve (se Figur 3.2). I stedet for at håndtere disse egenskaber separat for hvert legeme, er de pakket i et objekt. Det gøre det muligt at håndtere alle legemer i simuleringen ensartet.



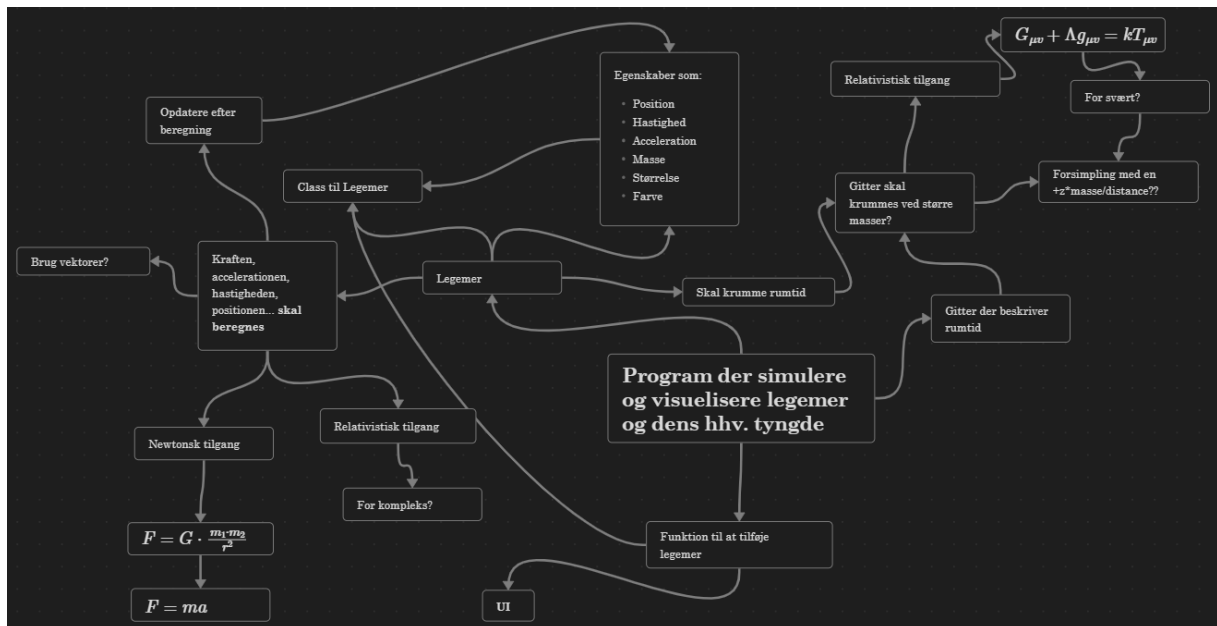
Figur 3.2: Et legeme i systemet har egenskaber som masse, størrelse, farve, position, hastighed og acceleration.

3.3 Decomposition

Dekomponering er en essentiel del af computational thinking for at få en struktureret tilgang til at håndtere problemer. Dette involvere at opdele programmets problem i mindre og mere håndterbare dele. Denne tilgang giver mulighed for at løse de opdelte mindre delproblemer, uden at blive overvældet af hele systemet på en gang. I forbindelse med simuleringen af gravitationen, er programmet dekomponeret i flere dele, der gøre koden mere struktureret, modulær og lettere at vedligeholde. Allerede i slutningen af indledningen begyndte dekomponeringen af programmet.

Grundlæggende set var programmets dekomponering baseret på mit eget rene mindflow³. Denne mindflow kan ses dokumenteret i den følgende mindmap i Figur 3.3.

3: Mindflow er progressionen eller udstrømning af ens tænkning, eller tanker; tankeproces.

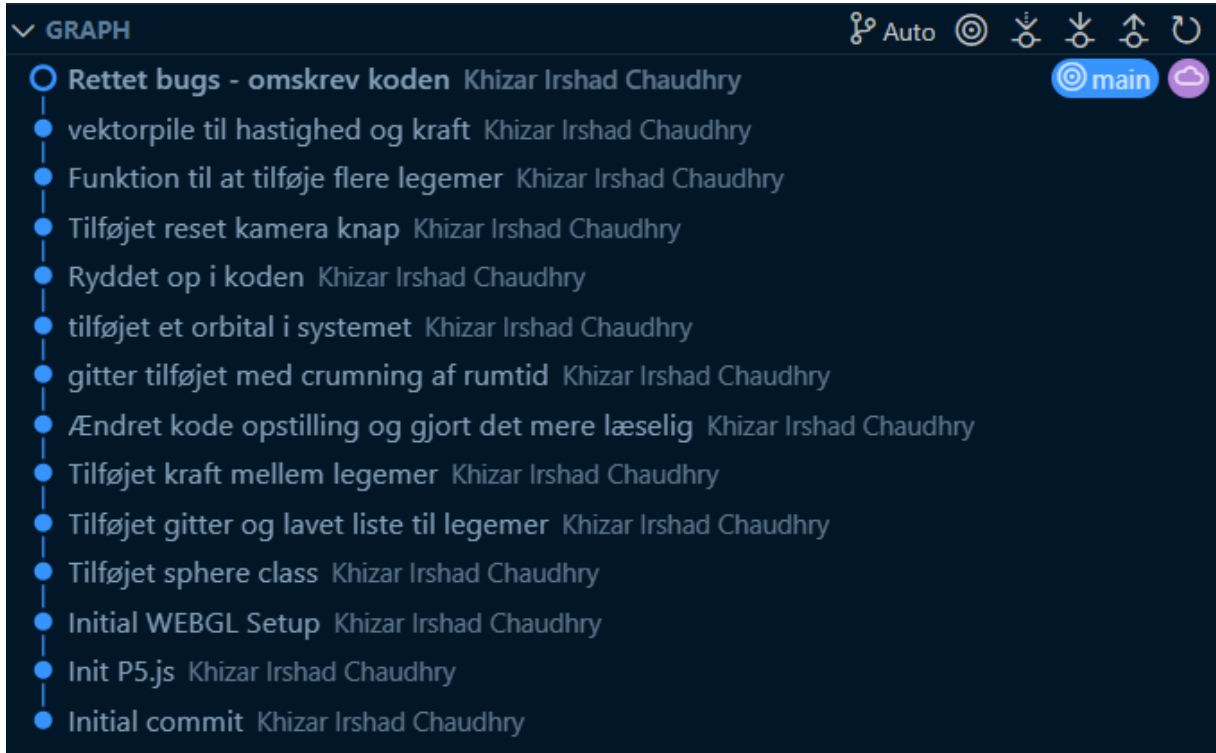


Figur 3.3: Mindmap over dekomponering af programmet i mindre overskuelige dele.

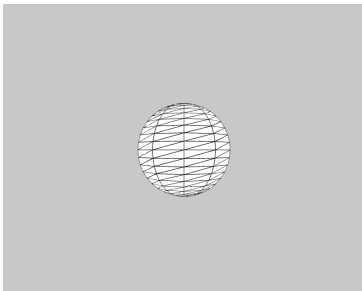
3.4 Algorithm Design

Algoritme design refererer til processen med at udvikle selve algoritmen til at løse det problemet. I softwareudvikling består algoritme design af at skabe de nødvendige metoder, funktioner og klasser, der gøre programmet funktionel. I denne program er algoritme design stortset det vigtigste for at programmet overhovedet fungerer. En detaljeret beskrivelse af programmets algoritme design repræsenteres i kapitel 5, *Dokumentation af Programmet*.

4 Udviklingsprocessen



Figur 4.1: Git-graph, der viser commits til remote repository origin/main.



Figur 4.2: Programmets første kugle - vha. object-oriented programmering.

1: P5.js er en JavaScript library, der bl.a. bruges til grafik

Projektets udviklingsprocess er godt og grundigt dokumenteret vha. version-kontrol systemet Git. I Figur 4.1 ses programmets Git-graph, der reflekterer en iterativ og systematisk tilgang til at udvikle programmet, hvor funktionalitet og struktur gradvist opbygges og forbedres. Denne afsnit redegøre for de væsentlige faser i udviklingen baseret på de forskellige commits i Git-graphen.

4.1 Intial commit & P5.js (WEBGL)

Den første commit markerer projektets start. Her blev projektet initialiseret, de grundlæggende filer blev tilføjet, og et P5.js¹ miljø blev sat op. En hurtig opsætning af WebGL-miljøet i P5.js blev også udført, for at muliggøre en 3D simulering.

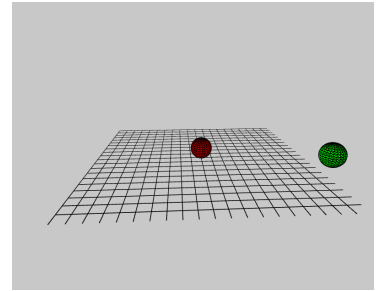
4.2 Tilføjet legeme klasse (OOP)

Her blev der tilføjet en klasse (OOP) til at håndtere de 3D-kugler, der repræsenterer legemerne i simuleringen (se Figur 4.2). Denne abstraktion muliggøre nem og overskuelig

håndtering af legemerne i systemet og deres egenskaber (masse, position, hastighed osv.). Mere om denne klasse kan læses i kapitel 5, Dokumentation af Programmet.

4.3 Tilføjet gitter og lavet liste til legemer

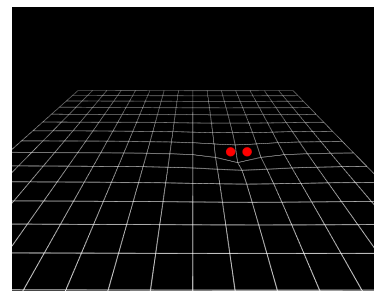
Et gitter blev tilføjet til simuleringen, der visualiserer rumtiden og hjælper intuitivt med at forstå deformationen af rummet, når legemerne interagerer med hinanden (se Figur 4.3). Der blev også lavet med alle legemer i systemet, til at holde styr på dem.



Figur 4.3: Implementering af et gitter, samt flere legemer i systemet.

4.4 Tilføjet tyngdekraften mellem legemer & krumning af rumtid

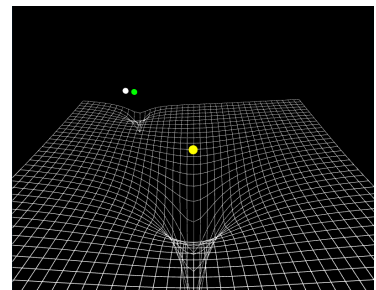
Den vigtigste del af simuleringen blev tilføjet, nemlig den gravitationelle attraktion mellem legemer, og ændringer deres hhv. positioner. Yderligere blev der implementeret visuel feedback for krumningen af rumtiden (en deformation i gitters z-aksen), som også er en vigtig del af simuleringen, især for at vise de relativistiske effekter. Resultatet af denne phase kan ses i Figur 4.4. Gennem denne phase blev der også justeret i selve programmets udseende og farver.



Figur 4.4: Implementationen af tyngdekraften, og dens påvirkning på rumtid gitteret.

4.5 Tilføjet et orbital i systemet

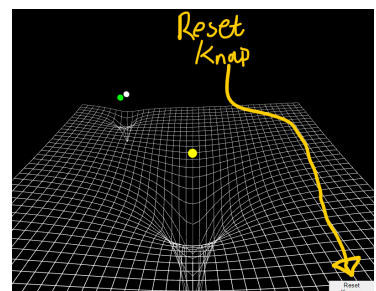
En orbital (bestående af tre legemer) blev tilføjet, som nogle *start-legemer*. Hovedsagligt for at teste om simuleringen virker so far. Orbitalets legemerenes begyndelsesbetænelser (start position og hastighed) blev stortset bestemt vha. bruteforcing. Orbitalernes funktionelle resultat gjorde det klart, at simuleringen givet et realistisk billed. For at tilføj noget unikhed ved legemerene, fik de forskellige farver. Dette kan ses i Figur 4.5. I denne phase blev programmets *need-to-have* krav fuldført. Efter denne phase blev der arbejdet på programmets *nice-to-have* krav.



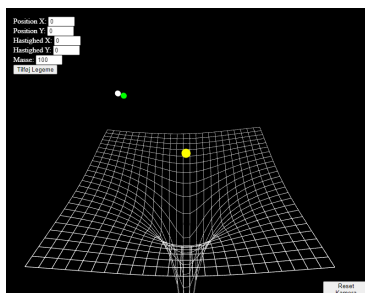
Figur 4.5: Implementering af orbitaldynamik og kredsløb for legemerne i simuleringen.

4.6 Tilføjet reset kamera knap

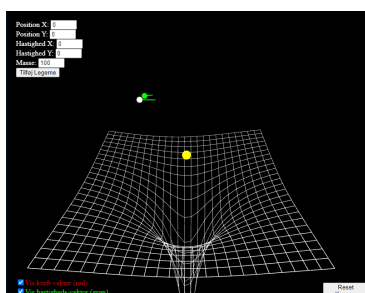
En funktionalitet til at nulstille kameraets position og vinkel blev tilføjet. Denne funktionalitet var især relevant, da man nemt kan komme til at rodde rundt i kameraets position og vinkel i WEBGL miljøet. Denne tilføjelse øger programmets user-experience (UX). Reset knappen kan ses i Figur 4.6.



Figur 4.6: Implementering af en knap til at reset kameraets position og vinkel.



Figur 4.7: Funktion samt user-interface (UI) til at tilføje legemer til selvvalgte positioner med selvvalgte hastigheder og masser



Figur 4.8: Funktion til at vise legemernes vektorer for hastighed og kraft som pile, samt user-interface (UI) til at tænde/slukke for funktionen.

4.7 Tilføjet funktion til at tilføje legemer

I denne fase blev der implementeret en funktion, samt en user-interface (UI) til at tilføje flere legemer. Dette gjorde simulationen langt mere interaktivt og engagerende for brugeren, da der nu kunne tilføjes legemere med i selvvalgte positioner med selvvalgte hastigheder og masser. Et billed af denne user-interface (UI) kan ses på Figur 4.7

4.8 Vektorpile til hastighed og kraft

Vektorene for hastighed og kraft blev visualiseret som pile (rettersagt linjer), som gør det lettere for at forstå de kræfter, der virker på legemerne og deres størrelse og retning. Samt hjælper det med at forstå påvirkningen på legemernes bevægelser. Der blev yderligere tilføjet funktionalitet til at tænde og slukke for visningen af disse vektorpile. Dette er en god eksempel på UX (user-experience), da det kan være at nogle bruger fortrækker ikke at se disse vektorpile. Denne funktionalitet kan ses i Figur 4.8.

Det er hermed her programmet blev erklæret som færdig.

Dokumentation af Programmet

5

Programmet simulerer gravitationen mellem flere legemer i et 3D-rum¹ vha. P5.js og WebGL. Programmet giver brugeren mulighed for at tilføje legemer, definere deres startbetingelser (position, hastighed og masse), og dernæst observere deres bevægelser under indflydelse af gravitation, samt deres påvirkning på rumtidsgitteret. Programmet visualiserer både legemernes bevægelser, deres indflydelse på rumtidsgitteret, og også de kræfter, der påvirker dem, i et dynamisk 3D-rum. I dette afsnit vil der gøres rede for programmets bagvedliggende logik og kode.

5.1 Hierarki chart for programmet

Programmet er opbygget med flere forskellige funktionelle enheder, der hver især håndterer specifikke (del)opgaver. Det er netop denne process der er præget af metodikken *abstraction fra computational thinking*. På Figur 5.1 ses disse funktionelle enheder, som forskellige filer, klasser og funktioner. De funktionelle enheder indeholder:

- **index.html**

Denne HTML fil, er programmets hovedfil. Det er netop denne fil der gengives i browseren. Det er i denne fil, hvor P5.js library og vores egen JavaScript logik implementeres som eksterne filer.

- **main.js**

Denne JavaScript fil er hovedfilen for hele programmets logik. Det er i denne fil simuleringen opsættes, beregnes og visualiseres.

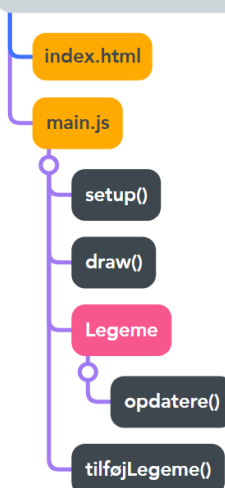
- **function setup()**

Dette er en P5.js funktion, der kører som det første i koden. Denne funktion køres kun en gang. I vores sammenhæng benyttes funktionen til at opsætte programmets UI og det grafiske miljø.

- **function draw()**

Dette er også en P5.js funktion, der kører hvert frame. Netop derfor benyttes denne funktion til at opdatere programmets tilstand. Det er i denne funktion koden opdatere og visualisere hele simuleringen.

Simulering af gravitation og visualisering af rumtid



Figur 5.1: På billedet ses hierarki chart for programmet, der giver et overblik over programmets filer og funktioner. Elementerne er farvet således, at gul repræsenterer filerne, sort repræsenterer funktioner/methods og pink repræsenterer klasser.

1: Selvom legemerne kun flyder i et 2-dimensionelt plane (x, y).

2: En method er en egenskab af en klasse, der indeholder en funktion.

► **class Legeme**

Denne klasse repræsenterer et legeme. Denne klasse har bl.a. en constructor, der opsætter legemets egenskaber som position, masse, hastighed, acceleration og farve.

► **method opdatere()**

Denne method² er tilknyttet til klassen *Legeme*, og benyttes til at bestemme et legemes hhv. position, hastighed og acceleration vha. tyngdekraftens påvirkning på den. Det er også i denne method, tyngdekraftens påvirkning bestemmes. Denne method tager et objekt (*legeme*) som parameter.

► **function tilføjLegeme()**

Denne funktion tilføjer legemer til systemet, på baggrund af brugerens input fra UI. Funktionen tilføjer en ny objekt af klassen *Legeme* til listen *legemer*.

5.2 Pseudokode

Pseudokode anvendes til at beskrive programmets logik, uden at være bundet til en bestemt syntaks. Pseudokode kan derfor være med til at formidle avanceret kodelogik og algoritmer, uden kendskab til et særlig programmeringssprog. Her er nogle af programmets funktioner og elementer beskrevet i pseudokode:

5.2.1 Pseudokode for Tilføjelse af Legeme

Når brugeren sender data (position, hastighed, masse) omkring et nyt legeme og klikket knappen *Tilføj legeme*, så aktiveres funktionen *tilføjLegeme*. Nedenfor er pseudokoden for denne funktion:

Funktion tilføjLegeme:

Modtag input fra bruger (position, hastighed, masse)

Tildele de numeriske inputs til variablerne (f.eks. x, y, v)

Opret et nyt objekt af klassen *Legeme* med de tildelte variable

Tilføj legeme til legemer (Array)

5.2.2 Pseudokode for opdatering af legemer

Legemernes positioner opdateres hvert frame. Dette sker vha. bestemmelse af legemets acceleration og hastighed, hvilket sker ved at bestemme den gravitationelle attraktion mellem legemerne. Alle disse beregninger sker netop i

Legeme klassens metode *opdatere*. Dette sker for hvert legeme.
Her er pseudokoden for denne funktion:

```
Funktion opdatere(legeme):
  Initialiser tidstrin (dt) til 0.5
  Nulstil acceleration til 0 (ingen kræfter virker i starten)

  For hvert legeme (andre) i alleLegemer:
    Hvis det nuværende legeme ikke er det samme som andre:
      Beregn retning af kraft mellem legeme og andre legeme
      Beregn afstand mellem legemerne (afstand^2)
      Beregn gravitationskraften ved hjælp af Newtons lov ( $F = G * m1 * m2 / r^2$ )
      Beregn accelerationen for det nuværende legeme ( $a = F / m$ )

  Opdater hastighed baseret på accelerationen ( $v = v + a * dt$ )
  Opdater position baseret på hastigheden ( $s = s + v * dt$ )
```

5.3 Kodestykker med indlejrede kommentarer

I denne sektion præsenteres den aktuelle kode, der står bag programmet. Koden er indlejrede med kommentarer, for at fortolke koden og gøre det nemmere at forstå.

5.3.1 Koden for opdatering af legemerne

Her er koden, der opdatere legemernes position og hastighed baseret på de kræfter, der virker på dem:

```
1 function opdatere(alleLegemer) {
2   let dt = 0.5; // Tidsinterval for opdatering af
   bevægelse
3   this.acceleration.set(0, 0); // Nulstil accelerationen
4
5   // Beregn kræfter mellem legemet og alle andre legemer
6   for (let andre of alleLegemer) {
7     if (andre !== this) { // Undgå at beregne kraften for
       legemet selv
8       let kraftRetning = p5.Vector.sub(andre.position, this
       .position); // Retningen af kraften mellem legemerne
9       let afstandSq = kraftRetning.magSq(); // Beregn
       kvadratet af afstanden mellem legemerne
10      let kraftStr = (G * this.masse * andre.masse) / (
       afstandSq + 100); // Beregn styrken af
       gravitationskraften
11      kraftRetning.normalize(); // Normaliser
       kraftretningen
12      let kraft = p5.Vector.mult(kraftRetning, kraftStr);
       // Beregn kraften som en vektor
13      this.acceleration.add(p5.Vector.div(kraft, this.masse
       )); // Beregn accelerationen af legemet
```

```

14     }
15 }
16
17 // Opdater hastighed og position
18 this.hastighed.add(p5.Vector.mult(this.acceleration, dt))
19   ; // Opdater hastigheden
20 this.position.add(p5.Vector.mult(this.hastighed, dt));
    // Opdater positionen
21 }

```

5.3.2 Visualisering af legemer og vektorpile

Nedenfor er koden, der håndterer visualiseringen af legemerne og deres tilkunttede vektorpile.

```

1 for (let legeme of legemer) {
2   push(); // Start en ny transformationsmatrix
3   fill(legeme.farve); // S t farven p legemet
4   translate(legeme.position.x, legeme.position.y, 0); //
    Placer legemet p dets position
5   sphere(10); // Tegn legemet som en 3D-sf re
6
7   strokeWeight(3); // S t tykkelsen p vektorpilene
8   // Tegn hastigheds-vektoren, hvis den er valgt af
    brugeren
9   if (visHastighedBox.checked()) {
10     stroke(0, 255, 0); // Gr n farve for hastigheds-
        vektoren
11     line(0, 0, 0, legeme.hastighed.x * 20, legeme.hastighed
        .y * 20, 0); // Tegn hastigheds-vektoren
12   }
13
14   // Tegn kraft-vektoren, hvis den er valgt af brugeren
15   if (visKraftBox.checked()) {
16     stroke(255, 0, 0); // R d farve for kraft-vektoren
17     line(0, 0, 0, legeme.acceleration.x * 400, legeme.
        acceleration.y * 400, 0); // Tegn kraft-vektoren
18   }
19   pop(); // Slut transformationen
20 }

```

5.4 Dokumantation af egne funktioner/klasse

Programmets kode er rig med egne funktioner og klasser. Et eksempel på det er den mest essentielle klasse *Legeme*.

Klassen *Legeme* repræsenterer et fysisk legeme, der kan bevæge sig under indflydelse af gravitationskræfter. Hver instans af klassen svarer til en kugle i simuleringen. Klassen håndterer legemets position, hastighed, acceleration og masse, samt opdatering af bevægelsen med metoden *opdater()* baseret på gravitationsinteraktioner med andre legemer i systemet.

Konstruktør: *Legeme(pos, masse, startHastighed, farve = [255, 0, 0])*

Parametre:

- ▶ *pos* (*p5.Vector*): Legemets startposition, givet som vektor.
- ▶ *masse* (*Float*): Legemets masse.
- ▶ *startHastighed* (*p5.Vector*): Legemets start-hastighed, givet som *p5.Vector*.
- ▶ *farve* (*Array*): Legemets farve. Standardværdi er [255, 0, 0] (rød).

Beskrivelse: Konstruktøren initialiserer legemet med det givet position, hastighed, masse og farve. Den sætter også den start accelerationen til nul.

Metode: *opdatere(alleLegemer)*

Parametre:

- ▶ *alleLegemer* (*Array*): En array af alle legemer i simuleringen.

Beskrivelse: Metoden *opdatere()* opdaterer legemets position og hastighed baseret på gravitationskræfterne, hvilket også bestemmes inde i metoden. Der bruges Newtons lov om gravitation til at beregne kraften og accelerationen.

Instansvariabler:

- ▶ `position` (`p5.Vector`): Legemets nuværende position i rummet. Det er et `p5.Vector`.
- ▶ `hastighed` (`p5.Vector`): Legemets nuværende hastighed (både retning og størrelse), repræsenteret som et `p5.Vector`.
- ▶ `acceleration` (`p5.Vector`): Legemets nuværende acceleration. I starten sat til nul, men opdateres i hver kald af `opdatere()`.
- ▶ `masse` (`Number`): Legemets masse, som bruges til bestemmelse af kræfter.
- ▶ `farve` (`Array`): Legemets farve, der bruges til visning i simuleringen.

5.5 Selveste koden & programmet

Selveste koden er uploadet på GitHub og kan tilgås på dette link:

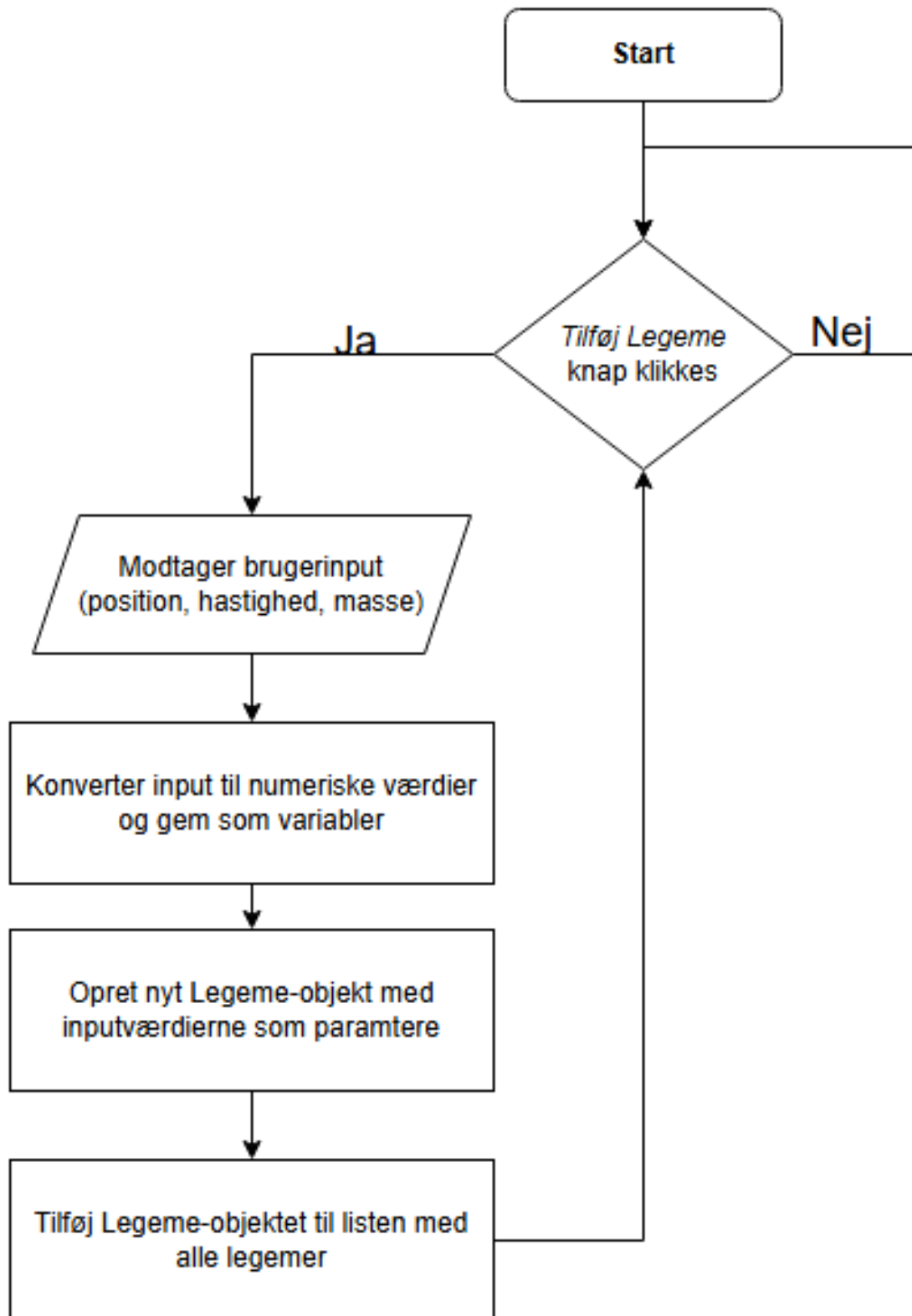
<https://github.com/KhizarIrshadChaudhry/GravitySimulation>

Selve programmet kan også tilgås og prøves, da den er hoset på GitHub pages. Programmet kan tilgås på dette link:

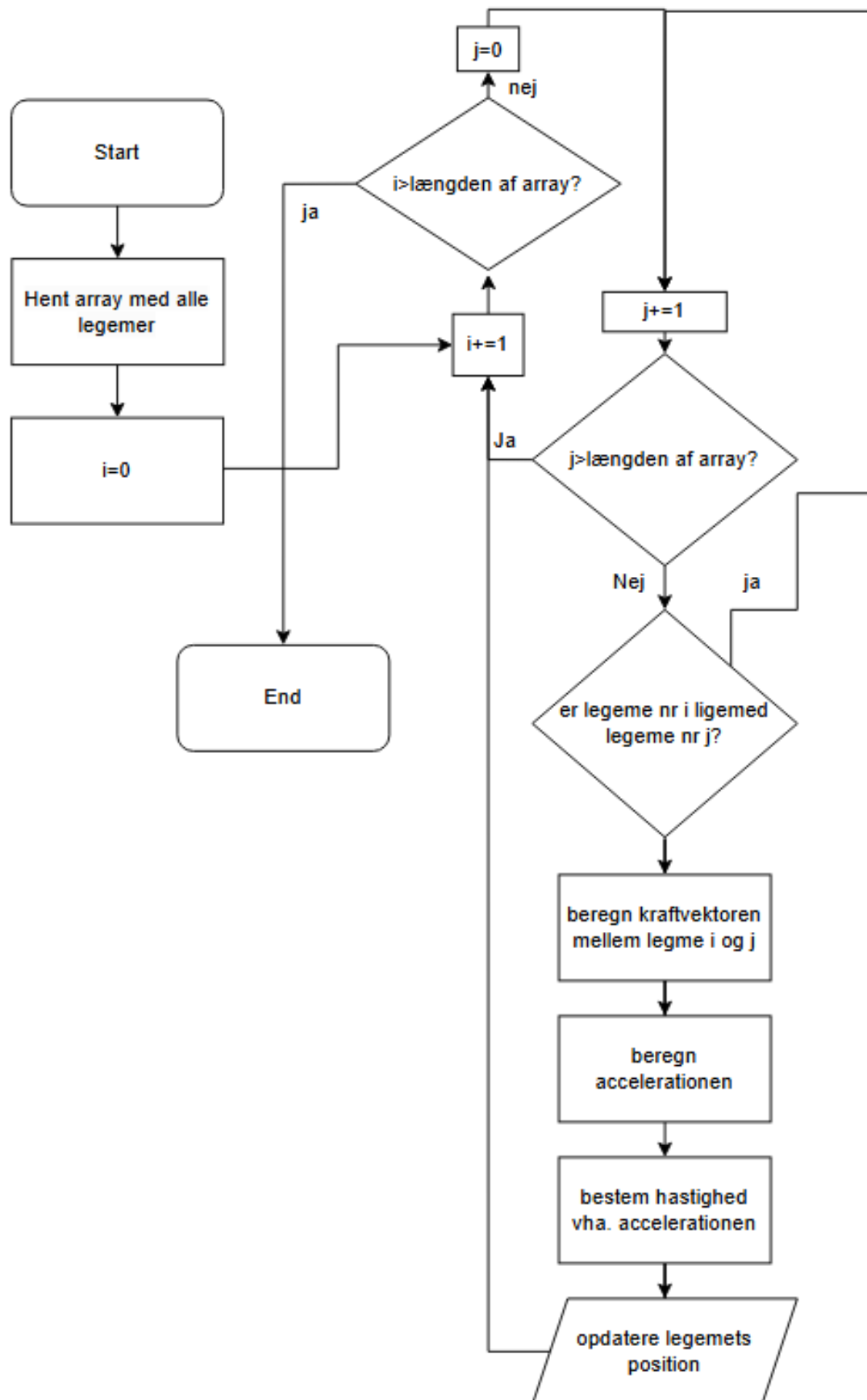
<https://khizarirshadchaudhry.github.io/GravitySimulation/>

5.6 Flowchart repræsentation af programmet

Flowcharts uddyber programmet, og illustrerer vigtige processer, selve logikken bag programmet. I dette program kan logikken bag væsentlige funktioner repræsenteres som flowcharts. Det kan f.eks. være funktioner som *tilføjLegeme()* (tilføjelse af legemer) og *opdatere()* (opdatering af legemernes position). Disse flowcharts kan ses på de næste to sider, i Figur 5.2 og 5.3



Figur 5.2: Flowdiagram over programmets funktionalitet omkring tilføjelse af legemer, baseret på data (position, masse, hastighed) fra brugeren.



Figur 5.3: Flowdiagram over opdatering af legemets position vha. accelerationen, som er forudsat af den gravitationelle attraktion mellem legemerne.

Testen af programmet bestod både af funktionelle tests og brugeroplevelsesteste. Det sikret at simuleringen fungerer korrekt, og ovenikøbet er brugervenlig.

► Funktionalitetstest

- **Oprettelse af legemer:**

Vi stede, at legemer kunne oprettes korrekt via brugernes inputfelter, og om de indgivet inputs stemmer ens med hvad der blev behandlet i kode-logikken. Ved at tilføje legemer til systemet, og tjekke værdierne i koden, kunne man bekræfte at inputfeltet virker korrekt.

- **Gravitationsberegning**

For at teste om simuleringen er korrekt, kigget man efter om bevægelserne overhovedet gav mening. Altså f.eks. om en stor masse rent faktisk tiltrækker og påvirker en mindre masses banse osv.

► Brugervenlighed

- **Performance:**

Man testede programmets performance, ved at tilføje enorm mange legemer, samt forstørrelse rum-tidsgitteret. Dette resulteret i et meget langsom, og ikke-reagerende program. Derfor man vi hurtig konstatere, at programmet hurtigt kan "*knækkes*", ved flere antal af legemer. Hvis programmet endag skulle udvides, bør dette problem være det første, der skal løses. Dette kan løses ved at benytte sig af mere optimerede og effektive algoritmer og metoder.

7

Diskussion

Udviklingen af programmet har været en sindsyg spændende og lærerig proces. I programmet er der klart styrker, men også områder, hvor der plads til forbedringer. Generelt har simuleringen opfyldt de grundlæggende krav, og jeg opnået de ønskende resultater i form af korrekt funktionalitet og en interaktiv user-experience (UX).

Et af de største udfordringer var klart, at forsimple Einsteins relativitetsteori, således at den kan simuleres. JavaScript er et relativt *high-level* programmeringssprog, netop derfor har det været ekstra svært at implementere relativitetsteorien i simulationen, uden at det sker på bekostning af for meget compile-tid og programmet stadig forholder sig responsivt.

Programmet kan dog stadig hurtigt blive *unresponsive*, når der er tilføjes endnu flere legemer, eller når rumtidens gitter udvides.

Programmet kunne udvides med flere funktioner, som f.eks. muligheden for at justere gravitationskonstanten, tilføje eksterne kræfter osv. Det kunne klart være en fordel at flytte simuleringen over et *low-level* language, for at igen inddrage relativistiske effekter. Dette kunne evt. gøres ved at bruge programmeringssproget C sammen med en library som GraphQL.

I forhold til brugeroplevelsen har programmet ikke haft mange komplikationer. UI'en har været intuitiv, og brugeren kunne nemt tilføje og interagere med legemer. Dog kunne der med fordel være indlagt et koordinatsystem, for at give en visuelrepræsentation af hvor legemerne ville *spawn* henne.

Samlet set er jeg tilfreds med resultatet, og programmet har opnået de ønskende funktionaliteter. Der er dog klart mulighed for både tekniske men også funktionelle forbedringer, der kunne gøre simuleringen mere effektiv og realistisk.

Udfra det opnået program, kan det konkluderes at, det ønskede mål blev opnået og vores opstillede problemformulering blev løst. Det opstillede problemformulering lød sådan:

Hvordan kan legemeres hhv. tyngdekraft simuleres og visualiseres vha. programmering?

Programmet formår at simulere gravitationelle interaktioner mellem legemer på en visuel og interaktive måde. Det gør den, da brugeren nemt kan tilføje legemer til systemet med selvvalgte begyndelsesbetingelser. Brugeren kan observere disse legemers bevægelser, og de pågældende krafter effekt på dem. Brugeren kan med fordel også observere legemernes påvirkning på rumtidens gitter.

Derfor fungerer simuleringen grundlæggende som forventet, og den giver en god visuel fremstilling af gravitationen. Dog er der plads til forbedringer, især hvad angår udvidelse og stabilitet. Dette kan evt. håndteres i fremtidige opdateringer og optimeringer.

Samlet set opfylder programmet sin funktionelle formål, og giver en enorm god brugeroplevelse, men der er potentiale for yderligere udvidelse og forfining.